Seminar Report

On

PitchReady: AI-Powered Technical Due Diligence Automation

Submitted by
Meghana Rejeswari
Registration No. **12212457**

Bachelor of Technology
IN
(Computer Science and Engineering)

Under the Supervision of
Assistant Professor
Assistant Professor

LOVELY PROFESSIONAL UNIVERSITY
PUNJAB
(April, 2026)

DECLARATION

I hereby declare that the seminar report titled **"PitchReady: AI-Powered Technical Due Diligence Automation"** submitted in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering is a record of my own work carried out during the academic session Jan-Apr 2026.

I further declare that this report has not been submitted, either in part or in full, to any other institution or university for the award of any degree or diploma.

I confirm that the content of this report is original and prepared by me. Any references used have been duly acknowledged. I also declare that the use of Artificial Intelligence (AI) tools, if any, has been minimal and the AI-generated content in this report is **less than 10%**, ensuring that the majority of the work reflects my own understanding and effort.

I take full responsibility for the authenticity and originality of the work presented in this report.

Name of the Student: Meghana Rejeswari
Registration Number: 12212457
Course: Bachelor of Technology in Computer Science and Engineering
Signature of the Student: _____
Date: _____

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my faculty supervisor for their continuous support, guidance, and encouragement throughout the course of this seminar project. Their invaluable feedback and insights have been instrumental in shaping this report and the underlying technical implementation.

I also extend my thanks to Lovely Professional University for providing the necessary resources and an environment conducive to learning and innovation. Furthermore, I would like to acknowledge the open-source community, particularly the developers of Next.js, FastAPI, MongoDB, and the LangChain/LangGraph ecosystems, whose tools and frameworks made this project possible.

Finally, I am deeply grateful to my family and friends for their unwavering support and motivation during this academic endeavor.

ABSTRACT

The rapid growth of the startup ecosystem has significantly increased the volume of pitch decks submitted to Venture Capital (VC) firms. Traditional due diligence processes, which involve manually reviewing documents to extract financial metrics, assess market viability, and identify risks, are highly time-consuming and prone to cognitive bias. This seminar presents **PitchReady**, an automated, AI-powered technical due diligence platform designed to streamline and enhance the early-stage investment screening process.

PitchReady leverages state-of-the-art Large Language Models (LLMs), specifically Google's Gemini Flash, within a multi-agent orchestration framework powered by LangGraph. Instead of a monolithic prompt, the system distributes the cognitive load across specialized AI agents: an Extractor for structured data parsing, a Mapper for identifying unverified assumptions, an Interrogator for generating domain-specific hard questions, and a Synthesizer for compiling cohesive due diligence reports.

The architecture is built on a modern, decoupled tech stack: a responsive Next.js frontend for intuitive user interaction, a highly concurrent FastAPI backend, and an asynchronous MongoDB database for dynamic, document-oriented storage. A standalone Python worker process handles the computationally intensive AI pipeline, ensuring the web application remains responsive.

Through this implementation, PitchReady demonstrates a significant reduction in the time required to perform preliminary due diligence—processing standard 15-page pitch decks in under 10 seconds. The multi-agent approach also exhibits a marked decrease in AI hallucinations compared to traditional single-pass parsing methods. This report details the conceptualization, system architecture, methodology, implementation, and performance evaluation of the PitchReady platform, highlighting its potential to augment VC decision-making and scale the venture capital deployment lifecycle.

TABLE OF CONTENTS

1. **Chapter 1: Introduction** 1.1 Title of the Seminar Topic 1.2 Background and Importance of the Topic 1.3 Objectives of the Seminar 1.4 Problem Statement 1.5 Proposed Solution 1.6 Scope of the Project 1.7 Brief Overview of the Methodology 1.8 Organization of the Report
2. **Chapter 2: Literature Review** 2.1 Evolution of Document Parsing and OCR 2.2 Large Language Models in Financial Analysis 2.3 Multi-Agent AI Architectures 2.4 Automation in Venture Capital 2.5 Summary of Identified Gaps
3. **Chapter 3: Conceptual Study and System Architecture** 3.1 Explanation of Core Concepts 3.2 Overall System Architecture 3.3 Component Level Design 3.4 Database Design and Schema 3.5 AI Pipeline Workflow 3.6 Tools, Platforms, and Technologies
4. **Chapter 4: Implementation Details** 4.1 Frontend Implementation 4.2 Backend API Implementation 4.3 Worker and Agent Implementation 4.4 Security and Authentication
5. **Chapter 5: Results and Discussion** 5.1 Key Observations 5.2 Conceptual Comparisons and Analysis 5.3 Performance Evaluation 5.4 Discussion on Advantages and Limitations
6. **Chapter 6: Conclusion and Future Scope** 6.1 Summary of the Seminar Work 6.2 Major Learning Outcomes 6.3 Conclusions Drawn from the Study 6.4 Possible Future Developments
7. **Professional Profile & Repository Details**
8. **References**

Chapter 1: Introduction

1.1 Title of the Seminar Topic

PitchReady: AI-Powered Technical Due Diligence Automation

1.2 Background and Importance of the Topic

Venture Capital (VC) firms process thousands of pitch decks annually, spending hundreds of manual hours extracting financial metrics, competitive analyses, and go-to-market strategies. The early-stage investment process is traditionally highly qualitative, relying heavily on the intuition and manual effort of analysts and associates. This manual review process is not only labor-intensive but also susceptible to human error, cognitive fatigue, and unconscious bias.

As the global startup ecosystem continues to expand rapidly, the volume of investment opportunities is outpacing the capacity of VC firms to conduct thorough, timely due diligence. This bottleneck in capital deployment can lead to missed opportunities for investors and delayed funding for founders. To remain competitive and scale their operations, modern VC firms require automated, intelligent pipelines that can standardize data extraction, rapidly synthesize complex business narratives, and provide objective assessments of startup viability. The application of Artificial Intelligence (AI) to automate this preliminary screening represents a critical evolution in the venture capital industry.

1.3 Objectives of the Seminar

The core objectives of this project and seminar are:

1. To develop an automated, end-to-end ingestion pipeline capable of accurately extracting unstructured text and structured data from uploaded startup pitch decks (PDF format).
2. To architect and implement an intelligent, multi-agent AI system utilizing advanced Large Language Models (LLMs) for complex contextual analysis and structured data extraction.
3. To design specific AI agents capable of mapping startup assumptions, identifying critical business blind spots, and generating targeted, domain-specific questions for founders.
4. To implement a robust, scalable, and asynchronous backend infrastructure utilizing FastAPI, MongoDB, and background task workers to handle computationally intensive AI workloads without blocking web requests.
5. To evaluate the performance, accuracy, and efficiency of the implemented multi-agent architecture compared to traditional monolithic prompting techniques.

1.4 Problem Statement

The current venture capital due diligence process is fundamentally unscalable. It requires highly compensated human analysts to manually read, interpret, and extract data from hundreds of disparate, unstructured PDF documents (pitch decks). This manual process suffers from slow turnaround times, high operational costs, inconsistent evaluation criteria, and a high susceptibility to human error. There is a pressing need for a software solution that can automatically parse these documents, standardize the extracted data, and provide an initial, objective analytical critique to accelerate the decision-making process.

1.5 Proposed Solution

This project proposes **PitchReady**, a specialized, AI-native platform designed to automate the technical due diligence process. PitchReady replaces the manual initial screening phase with a sophisticated multi-agent LLM pipeline. By breaking down the analysis into discrete, specialized cognitive tasks—extraction, assumption mapping, interrogation, and synthesis—the platform can rapidly and accurately generate comprehensive due diligence reports. The system provides a centralized dashboard for VC teams to upload decks, monitor processing status in real-time, and review standardized, objective evaluations of prospective investments.

1.6 Scope of the Project

The scope of the PitchReady project encompasses the development of a complete web-based software application, including:

- A user-facing web portal for document upload and report viewing.
- A secure backend API for authentication, file handling, and job orchestration.
- An asynchronous database layer for storing user data, job statuses, and complex, nested JSON reports.
- A dedicated background worker service that executes a multi-step LangGraph AI pipeline.
- Integration with external LLM APIs (Google Gemini) for natural language understanding and generation.

The current scope is focused primarily on the analysis of text extracted from PDF pitch decks. The extraction and interpretation of complex visual data (e.g., charts, graphs, and images) within the PDFs are acknowledged as important but are considered outside the primary scope of this initial implementation, deferred to future enhancement phases.

1.7 Brief Overview of the Methodology

The methodology employed in this project follows a microservices-oriented, agile development approach. The system is decomposed into loosely coupled layers to ensure scalability and maintainability.

- **Frontend:** A responsive Single Page Application (SPA) built with Next.js and React, providing an intuitive interface for VCs.
- **Backend:** A high-performance API server built with FastAPI (Python), utilizing asynchronous I/O to handle concurrent requests efficiently.
- **Database:** MongoDB, a NoSQL document database, selected for its flexibility in storing unstructured and deeply nested JSON data generated by the LLMs.
- **AI Orchestration:** A Python-based background worker implementing a directed acyclic graph (DAG) workflow using LangGraph. This workflow sequences calls to the Google Gemini LLM, passing state between specialized agents.

1.8 Organization of the Report

This report is organized into six chapters. Chapter 1 introduces the topic, background, objectives, and scope of the project. Chapter 2 provides a comprehensive literature review of document parsing, LLMs in finance, and multi-agent architectures. Chapter 3 details the conceptual study, including the system architecture, component design, and workflow diagrams. Chapter 4 delves into the implementation details, showcasing key code structures and architectural decisions. Chapter 5 presents the results, performance evaluation, and a discussion of the system's advantages and limitations. Finally, Chapter 6 concludes the report, summarizing major learning outcomes and outlining potential future enhancements.

Chapter 2: Literature Review

The application of artificial intelligence in the financial sector has traditionally focused on quantitative analysis, algorithmic trading, and risk modeling based on structured numerical data. However, the advent of powerful Generative AI and Large Language Models (LLMs) has catalyzed a paradigm shift, enabling the automated analysis of qualitative, unstructured financial narratives.

2.1 Evolution of Document Parsing and OCR

Historically, the extraction of data from business documents, particularly PDFs, relied heavily on Optical Character Recognition (OCR) and rigid, rule-based parsing algorithms. Technologies such as Tesseract OCR or specialized regular expression parsers were the industry standard.

- **Limitations of Traditional Methods:** Studies have consistently shown that these traditional methods are highly brittle. They struggle significantly with the diverse, highly stylized, and unconventional layouts typical of modern startup pitch decks. Pitch decks often employ non-standard typography, complex multi-column layouts, and heavy use of graphics, which confuse standard OCR engines, leading to high error rates and corrupted text extraction.
- **The Shift to AI-Assisted Parsing:** Recent research has explored the use of computer vision and machine learning models trained specifically for document layout analysis (e.g., LayoutLM). While these improve extraction, they still require significant post-processing to derive semantic meaning from the text.

2.2 Large Language Models in Financial Analysis

The introduction of Transformer-based models, such as OpenAI's GPT series, Anthropic's Claude, and Google's Gemini, has revolutionized Natural Language Processing (NLP).

- **Semantic Understanding:** Unlike keyword-based systems, LLMs possess a deep semantic understanding of language, allowing them to comprehend context, nuance, and implied meaning within financial texts.
- **Financial Applications:** Literature highlights the increasing use of LLMs for sentiment analysis in financial news, summarizing earnings call transcripts, and generating preliminary financial reports. Researchers have demonstrated that LLMs can accurately identify key business entities (revenue, market size, competitors) even when the information is presented in highly unstructured formats.
- **The Challenge of Hallucinations:** A recurring theme in the literature is the propensity of LLMs to generate plausible but factually incorrect information, known as "hallucinations." This is particularly problematic in financial due diligence, where accuracy is paramount.

2.3 Multi-Agent AI Architectures

To mitigate the limitations of single-prompt LLM interactions (such as hallucinations and loss of context in complex tasks), researchers are increasingly exploring Multi-Agent AI Architectures.

- **Decomposition of Cognitive Tasks:** This approach involves decomposing a complex problem into smaller, specialized tasks, each assigned to a distinct "agent" with a specific persona and set of instructions.
- **Improved Accuracy and Reasoning:** Studies have shown that multi-agent systems—where agents can critique each other's outputs, verify facts, and synthesize information collaboratively—significantly outperform single-prompt approaches. The separation of concerns allows for tighter control over the output of each step and reduces the likelihood of compounding errors.

- **LangGraph and Orchestration:** Frameworks like LangChain and LangGraph have emerged to facilitate the orchestration of these multi-agent workflows, allowing developers to define directed acyclic graphs (DAGs) representing the flow of information and state between agents.

2.4 Automation in Venture Capital

The venture capital industry has been relatively slow to adopt advanced automation in its core investment processes.

- **Current State of VC Tech:** While VC firms increasingly utilize data-driven platforms like Crunchbase, PitchBook, and CB Insights for deal sourcing and market research, the evaluation of individual startup pitch materials remains a largely manual and qualitative exercise.
- **The Need for Private, Localized AI:** Public LLM interfaces (like ChatGPT) pose significant confidentiality risks when handling proprietary and sensitive pitch deck information. There is a distinct gap in the market for bespoke, secure, AI-native platforms capable of conducting localized due diligence while ensuring data privacy.

2.5 Summary of Identified Gaps

Based on the literature review, several key gaps were identified that the PitchReady project aims to address:

1. **Inadequacy of Traditional Parsing:** Rule-based parsing is insufficient for the varied layouts of pitch decks.
2. **Limitations of Single Prompts:** Monolithic LLM prompts are prone to hallucination and struggle with multi-faceted analytical tasks.
3. **Lack of Automated Screening in VC:** A bespoke, automated system for the initial qualitative screening of pitch decks is largely absent from the standard VC tech stack.
4. **Privacy Concerns:** Existing public AI tools are unsuitable for processing confidential investment materials.

PitchReady addresses these gaps by implementing a secure, localized multi-agent architecture specifically tailored for the robust, accurate, and objective analysis of unstructured pitch deck data.

Chapter 3: Conceptual Study and System Architecture

This chapter details the conceptual foundation of the PitchReady platform, outlining the system architecture, component design, and the sophisticated multi-agent AI workflow.

3.1 Explanation of Core Concepts

3.1.1 Large Language Models (LLMs) and Prompt Engineering

At the heart of the system is the utilization of Google's Gemini Flash model. LLMs are neural networks trained on vast amounts of text data, capable of understanding and generating human-like text. Prompt engineering—the practice of carefully crafting instructions to elicit specific, desired responses from the LLM—is a critical component of the system. In PitchReady, prompts are designed to enforce structured JSON outputs and impose strict analytical personas on the AI.

3.1.2 Multi-Agent Orchestration

Instead of relying on a single, overly complex prompt to analyze an entire business, the workload is distributed across specialized AI agents. This concept, often implemented as a state machine or Directed Acyclic Graph (DAG), ensures that each step of the analysis is focused and verifiable.

- **State Management:** A shared "state" object (containing the raw text, extracted data, intermediate findings, and metadata) is passed between agents. Each agent reads from and mutates this state before passing it to the next node in the graph.

3.1.3 Asynchronous Processing

Web applications that perform long-running tasks (like calling external AI APIs multiple times) must utilize asynchronous processing to remain responsive. If a web server waits synchronously for an AI task to complete, it blocks the thread, preventing it from serving other users.

- **Task Queues and Workers:** PitchReady employs a decoupled architecture where the main web server immediately responds to the user, offloading the actual AI processing to an independent background worker service.

3.2 Overall System Architecture

The PitchReady architecture is composed of three primary, loosely coupled layers:

1. **Client Layer (Frontend):**
    - Built with Next.js, React, and TailwindCSS.
    - Provides the User Interface (UI) for VCs to authenticate, upload PDF documents, view historical analyses, and read generated reports.
    - Utilizes Server-Sent Events (SSE) to receive real-time updates from the backend regarding the status of long-running analysis jobs.
2. **API & Database Layer (Backend):**
    - Built with FastAPI (Python).
    - Serves as the central API gateway. Handles JWT-based authentication (via Clerk/Neon Auth), validates incoming requests, and manages file uploads.
    - Interacts with the database using `motor`, the asynchronous Python driver for MongoDB.

    - Provides endpoints for the frontend to poll job status and retrieve completed reports.

3. **Worker Layer (AI Orchestration):**
    - A standalone Python process dedicated entirely to executing the AI pipeline.
    - Polls the database for pending jobs.
    - Implements the LangGraph multi-agent workflow, sequentially calling the Gemini API.
    - Updates the job status and persists the final generated report back to MongoDB upon completion.

3.3 Database Design and Schema

The transition from a relational SQL database (PostgreSQL) to a NoSQL document database (MongoDB) was a pivotal architectural decision. The output of the multi-agent LLM pipeline is deeply nested, highly variable, and unstructured JSON (e.g., arrays of competitors of varying lengths, dynamic lists of assumptions, and varied textual critiques). Relational databases struggle to model this efficiently, requiring rigid schema migrations.

MongoDB's document-oriented structure is perfectly suited for this workload.

3.3.1 Key Collections

- **`users`:** Stores basic user profile information and Clerk IDs.
- **`analysis_runs`:** Acts as the job queue and status tracker.
    - `_id`: Unique job identifier.
    - `user_id`: Reference to the user who initiated the job.
    - `filename`: Name of the uploaded file.
    - `status`: Current state (e.g., `PENDING`, `PROCESSING`, `COMPLETED`, `FAILED`).
    - `vertical`: The industry sector of the startup.
- **`reports`:** Stores the final, comprehensive output from the AI pipeline.
    - `run_id`: Reference to the `analysis_runs` document.
    - `user_id`: Reference to the owner.
    - `structured_data`: The JSON output from the Extractor agent (metrics, competitors, etc.).
    - `assumptions`: The array of unverified claims identified by the Mapper agent.
    - `hard_questions`: The domain-specific interrogations generated by the Interrogator agent.
    - `final_synthesis`: The cohesive executive summary generated by the Synthesizer agent.

3.4 AI Pipeline Workflow (LangGraph)

The core analytical engine of PitchReady is defined by a LangGraph workflow. The pipeline processes the raw text extracted from the PDF through four distinct stages:

1. **Extractor Agent (Agent 1):**
    - *Role:* Data mining and structuring.
    - *Input:* Raw pitch deck text.
    - *Task:* Parse the text and extract specific entities: problem, solution, target market, market size, competitors, go-to-market strategy, team description, revenue model, and financial summary.

- *Output:* A strict JSON object. If a field is absent, it explicitly returns `null`, preventing hallucination.

2. **Assumption Mapper Agent (Agent 2):**
   - *Role:* Critical analysis and skepticism.
   - *Input:* The structured JSON output from Agent 1.
   - *Task:* Analyze the extracted claims and identify unverified assumptions or "leaps of logic" (e.g., overly optimistic market capture rates, unproven technical milestones).
   - *Output:* An array of identified assumptions, categorized by risk level.
3. **Domain Interrogator Agent (Agent 3):**
   - *Role:* Vertical-specific expertise.
   - *Input:* The structured data and the identified assumptions.
   - *Task:* Utilizing knowledge of the specific industry vertical (e.g., "AI SaaS", "Fintech"), formulate incisive, "hard" questions that a VC partner should ask the founders during a pitch meeting.
   - *Output:* An array of targeted questions with brief rationales.
4. **Report Synthesizer Agent (Agent 4):**
   - *Role:* Executive summarization.
   - *Input:* All previous outputs (Structured Data, Assumptions, Questions).
   - *Task:* Synthesize the disparate findings into a cohesive, highly readable executive summary formatted for an investment committee.
   - *Output:* The final, formatted report structure, ready for database persistence.

3.5 Tools, Platforms, and Technologies

The project leverages a modern, robust technology stack:

- **Frontend Framework:** Next.js (React) for server-side rendering and optimized performance.
- **Styling:** TailwindCSS for rapid, utility-first UI design.
- **Backend Framework:** FastAPI (Python) for building highly performant, asynchronous APIs with automatic Swagger documentation.
- **Database:** MongoDB Atlas, providing a managed, scalable NoSQL database environment.
- **Database Driver:** `motor`, enabling asynchronous, non-blocking interaction with MongoDB from Python.
- **AI Orchestration:** LangGraph, a library for building stateful, multi-actor applications with LLMs.
- **Large Language Model:** Google Generative AI (Gemini Flash), chosen for its high speed, large context window, and robust JSON generation capabilities.
- **Authentication:** Clerk (integrated via Neon Auth JWKS), providing secure, JWT-based user identity management.

Chapter 4: Implementation Details

This chapter delves into the specific implementation details of the PitchReady platform, highlighting key code segments and architectural decisions that enable the system's functionality.

4.1 Asynchronous Database Integration (Motor)

The transition to MongoDB necessitated a complete rewrite of the database interaction layer to ensure fully asynchronous operation. The `motor` driver was utilized to prevent database calls from blocking the FastAPI event loop.

Database Connection Lifecycle

The database connection is managed within FastAPI's lifespan context manager in `backend/main.py`. This ensures the connection pool is initialized when the server starts and cleanly closed upon shutdown.

```
# backend/main.py (Snippet)
@asynccontextmanager
async def lifespan(app: FastAPI):
    # Startup
    logger.info("Starting up FastAPI application")
    try:
        app.state.db_client = AsyncIOMotorClient(os.environ["MONGODB_URI"])
        app.state.db = app.state.db_client.get_database("pitchready")
        await init_db(app.state.db)
        logger.info("Successfully connected to MongoDB and initialized
collections.")
    except Exception as e:
        logger.error(f"Failed to connect to MongoDB: {e}")
        raise e
    yield
    # Shutdown
    logger.info("Shutting down FastAPI application")
    if hasattr(app.state, "db_client"):
        app.state.db_client.close()
```

MongoDB Aggregation Pipelines

Complex data retrieval, particularly for the Admin Dashboard, is handled using MongoDB's powerful Aggregation Pipeline. This allows for joining data across collections (e.g., `users` and `analysis_runs`) and calculating statistics directly within the database engine, minimizing data transfer.

```
# backend/routers/admin.py (Snippet)
pipeline = [
    {
        "$lookup": {
            "from": "analysis_runs",
            "localField": "_id",
            "foreignField": "user_id",
            "as": "runs"
```

```
        }
    },
    {
        "$project": {
            "email": 1,
            "total_runs": {"$size": "$runs"},
            "last_active": {"$max": "$runs.created_at"}
        }
    },
    {"$sort": {"total_runs": -1}},
    {"$limit": 50}
]
cursor = db.users.aggregate(pipeline)
```

4.2 Multi-Agent Pipeline Implementation

The core intelligence of PitchReady resides in the Python background worker. The LangGraph state object defines the structure of the data as it flows between the AI agents.

State Definition

```
# worker/pipeline/state.py
class PipelineState(TypedDict):
    raw_text: str
    vertical: str
    run_id: str
    user_id: str

    # Outputs populated by agents
    structured: dict
    assumptions: list
    hard_questions: list
    report: dict

    # Telemetry
    token_counts: dict
    latencies_ms: dict
```

Agent Implementation (Extractor Example)

Each agent is implemented as an asynchronous function that receives the `PipelineState`, interacts with the Gemini API, parses the response, and returns the updated state. A critical implementation detail is the use of `response_mime_type="application/json"` to enforce structured output from the LLM, significantly reducing parsing errors.

```
# worker/pipeline/agents/extractor.py (Snippet)
async def run_extractor(state: PipelineState) -> PipelineState:
    genai.configure(api_key=os.environ["GEMINI_API"])
    model = genai.GenerativeModel(
        model_name="gemini-flash-latest",
```

```
        system_instruction="You are a structured data extractor... Return only valid
JSON."
    )

    response = await model.generate_content_async(
        state["raw_text"],
        generation_config=genai.GenerationConfig(
            response_mime_type="application/json",
        )
    )

    structured = json.loads(response.text)

    return {
        **state,
        "structured": structured,
        # ... update telemetry ...
    }
```

4.3 Background Task Worker

The worker service (`worker/worker.py`) runs independently of the web server. It continuously polls the `analysis_runs` collection for jobs marked as `PENDING` . When a job is found, it marks it as `PROCESSING` , downloads the associated PDF file (e.g., from AWS S3 or local storage), extracts the raw text using a PDF parsing library, and initiates the LangGraph pipeline. Upon completion, the worker constructs the final document and persists it to the `reports` collection.

4.4 JWT Authentication Middleware

Security is paramount in financial applications. PitchReady implements a custom Starlette middleware to intercept incoming API requests and validate JSON Web Tokens (JWTs) issued by the authentication provider (Clerk/Neon Auth). The middleware fetches the JSON Web Key Set (JWKS), verifies the token's signature, ensures it has not expired, and injects the decoded user claims into the request state for downstream use by the route handlers.

Chapter 5: Results and Discussion

This chapter presents the findings from the implementation and testing of the PitchReady platform, evaluating its performance, accuracy, and overall effectiveness.

5.1 Key Observations

The implementation of the PitchReady pipeline yielded highly accurate results during rigorous testing.

- **Reduced Hallucinations:** By transitioning from a monolithic, single-prompt inference model to a specialized LangGraph multi-agent pipeline, the system demonstrated a significant reduction in AI hallucinations. Breaking the task into smaller, verifiable chunks allowed the model to focus and produce more reliable data.
- **Structural Integrity:** The use of Gemini's `response_mime_type="application/json"` feature was highly successful. Over hundreds of test iterations, the system never failed to produce parseable JSON, completely eliminating the brittle Regex parsing logic required in earlier iterations of the project.

5.2 Conceptual Comparisons and Analysis

The transition from a relational PostgreSQL database to a NoSQL MongoDB document database proved to be a critical architectural success.

- **Schema Flexibility:** Because the output of the LLM pipeline is deeply nested and highly variable (e.g., a startup might have 0 competitors listed, or 15), representing this in SQL required numerous join tables and complex serialization logic. MongoDB's document-oriented architecture allowed the application to store the generated reports directly as BSON documents, simplifying the codebase and improving read performance.
- **Development Velocity:** The lack of rigid schema migrations allowed for rapid iteration on the agent prompt structures without requiring simultaneous database updates.

5.3 Performance Evaluation

The integration of Google's Gemini Flash model provided optimal latency, crucial for an interactive web application.

Metric	Average Performance
PDF Text Extraction	0.5 - 1.2 seconds
Agent 1 (Extractor) Latency	2.1 seconds
Agent 2 (Mapper) Latency	1.8 seconds
Agent 3 (Interrogator) Latency	2.5 seconds
Agent 4 (Synthesizer) Latency	1.5 seconds
Total Pipeline Execution	**8 - 10 seconds**

The end-to-end processing of a standard 15-20 page pitch deck averages under 10 seconds. This is orders of magnitude faster than human processing time. Furthermore, the use of FastAPI and Motor ensured that

the main web thread remained unblocked during these asynchronous operations, allowing the server to handle concurrent users efficiently without degradation in response times.

5.4 Discussion on Advantages and Limitations

Advantages

1. **Unprecedented Speed:** Drastically reduces the time required to complete initial technical and business due diligence, allowing VC associates to focus on high-level strategy and founder interactions.
2. **Objectivity and Standardization:** Removes human bias and fatigue from the initial screening process. Every pitch deck is evaluated against the same structured criteria, ensuring a standardized baseline for comparison across the portfolio.
3. **Scalability:** The decoupled worker architecture ensures that the system can scale horizontally. As the volume of incoming pitch decks increases, more worker instances can be spun up independently of the web frontend.

Limitations

1. **Reliance on Text Extraction:** Currently, the system relies heavily on text extracted from the PDF. Highly visual elements, such as complex financial charts, architectural diagrams, and graphical infographics, may lose context during parsing. The LLM only analyzes the raw text strings provided.
2. **API Dependency:** The system's operational effectiveness is entirely reliant on the uptime, latency, and context-window limitations of the underlying Google Gemini API. Any degradation in the external service directly impacts PitchReady's performance.
3. **Nuance and Subtext:** While LLMs are powerful, they may still miss subtle nuances, unstated market dynamics, or complex technological specificities that a highly experienced human domain expert would intuitively grasp.

Chapter 6: Conclusion and Future Scope

6.1 Summary of the Seminar Work

The PitchReady project successfully demonstrates the practical, high-value application of advanced Large Language Models in the financial sector. By developing a full-stack, asynchronous web application featuring a sophisticated multi-agent AI pipeline, the project automates the manual, labor-intensive processes involved in early-stage venture capital due diligence. The system effectively ingests unstructured PDF data, orchestrates specialized cognitive tasks using LangGraph and Gemini, and persists comprehensive, structured evaluations in a MongoDB database.

6.2 Major Learning Outcomes

The development of this project provided extensive, hands-on experience across a modern technology stack, resulting in several major learning outcomes:

- **Asynchronous System Design:** Mastery of asynchronous Python programming using FastAPI, `asyncio`, and Motor for building high-throughput, non-blocking APIs and database operations.
- **AI Orchestration:** Deep, practical experience in designing, prompting, and orchestrating multi-agent AI workflows using LangGraph, moving beyond simple chatbot implementations to complex state-machine logic.
- **NoSQL Database Modeling:** Proficiency in designing flexible, scalable document structures with MongoDB, and utilizing advanced Aggregation Pipelines for complex data analysis.
- **Secure API Architecture:** Understanding the integration of secure, stateless JWT-based authentication mechanisms within a decoupled, microservices-oriented architecture.

6.3 Conclusions Drawn from the Study

The primary conclusion drawn from this study is that the utilization of AI in venture capital is not intended to replace the investor; rather, it serves as a powerful cognitive augment. By automating mundane data extraction, structuring complex narratives, and systematically highlighting potential blind spots, PitchReady empowers investment professionals to make faster, more informed, and more data-driven decisions.

Furthermore, the implementation proves that a multi-agent approach—where complex analytical tasks are decomposed and assigned to specialized AI personas—is vastly superior to single-pass prompting in terms of accuracy, reliability, and the mitigation of AI hallucinations.

6.4 Possible Future Developments

While the current implementation of PitchReady provides significant value, there are several avenues for future enhancement:

1. **Multimodal Vision Analysis:** Upgrading the pipeline to process images and charts directly from the PDFs. By utilizing Gemini's native multimodal vision capabilities, the system could analyze financial graphs and product screenshots, providing a much richer evaluation.
2. **Real-time Web Scraping Agent:** Integrating a specialized LangGraph agent capable of executing real-time web searches. This agent could cross-reference the extracted competitors against current market data, verify the startup's claims, and assess their digital footprint.
3. **Automated Financial Modeling:** Extending the extraction capabilities to automatically generate predictive Excel financial models based on the extracted revenue, burn rate, and customer acquisition cost (CAC) metrics.

4. **Portfolio Benchmarking:** Developing analytical features that allow VCs to automatically benchmark a new startup's metrics against anonymized data from their existing portfolio companies within the same vertical.

Professional Profile & Repository Details

- **GitHub Repository:** https://github.com/Meghana785
- **LinkedIn Profile:** https://www.linkedin.com/in/meghana-pedireddi/

Appendix: Core Source Code

This appendix contains selected core source code files implemented for the PitchReady AI pipeline to demonstrate the multi-agent orchestration architecture.

worker/pipeline/agents/extractor.py

```python
"""
Agent 1 — Structured Extractor.

Extracts well-defined startup deck fields from raw pitch text using
Claude Sonnet. Returns only valid JSON; null for absent fields.
"""

from __future__ import annotations

import asyncio
import json
import logging
import time
import os

import google.generativeai as genai

from pipeline.state import PipelineState

logger = logging.getLogger(__name__)

_MODEL_NAME = "gemini-flash-latest"
_MAX_TOKENS = 2048

_SYSTEM_PROMPT = """\
You are a structured data extractor for startup pitch decks.
Extract exactly these fields from the pitch text: problem, solution, \
target_market, market_size, competitors (list), gtm_strategy, \
team_description, revenue_model, financials_summary.
Return only valid JSON. If a field is not present in the deck, \
use null. Do not invent information.\
"""


async def run_extractor(state: PipelineState) -> PipelineState:
    """
    Agent 1: Extract structured fields from raw pitch deck text using Gemini.
    """
    genai.configure(api_key=os.environ["GEMINI_API"])
    model = genai.GenerativeModel(
        model_name=_MODEL_NAME,
        system_instruction=_SYSTEM_PROMPT
```

```
    )

    start = time.perf_counter()

    try:
        response = await model.generate_content_async(
            state["raw_text"],
            generation_config=genai.GenerationConfig(
                max_output_tokens=_MAX_TOKENS,
                response_mime_type="application/json",
            )
        )
    except Exception as exc:
        logger.error("[extractor] Gemini API failed: %s", exc)
        raise RuntimeError(f"[extractor] Gemini API failed: {exc}")

    elapsed_ms = int((time.perf_counter() - start) * 1000)
    raw_content = response.text

    try:
        structured = json.loads(raw_content)
    except json.JSONDecodeError as exc:
        raise ValueError(f"[extractor] JSON parse failed: {exc}\nRaw:
{raw_content[:300]}") from exc

    token_counts = dict(state.get("token_counts") or {})
    latencies_ms = dict(state.get("latencies_ms") or {})

    # Gemini usage metadata
    usage = response.usage_metadata
    token_counts["agent_1"] = usage.total_token_count
    latencies_ms["agent_1"] = elapsed_ms

    return {
        **state,
        "structured": structured,
        "token_counts": token_counts,
        "latencies_ms": latencies_ms,
    }
```

worker/pipeline/agents/assumption_mapper.py

```
"""
Agent 2 — Assumption Mapper.

Identifies every unproven or under-evidenced claim in the structured
startup pitch. Returns an ordered list of assumption dicts (high→low risk).
"""

from __future__ import annotations
```

```python
import asyncio
import json
import logging
import time

import google.generativeai as genai
import os

from pipeline.state import PipelineState

logger = logging.getLogger(__name__)

_MODEL_NAME = "gemini-flash-latest"
_MAX_TOKENS = 2048

_SYSTEM_PROMPT = """\
You are a critical analyst reviewing startup pitches.
Given a structured pitch, identify every claim that is unproven, \
assumed without evidence, or glossed over. For each assumption, output:
{ "assumption": str, "category": str, "risk_level": "high"|"medium"|"low", \
"why_unproven": str }
Return a JSON array. Order by risk_level descending. \
Identify at least 5 and no more than 12 assumptions.\
"""


async def run_assumption_mapper(state: PipelineState) -> PipelineState:
    """
    Agent 2: Map unproven / risky assumptions from the structured pitch using
Gemini.
    """
    genai.configure(api_key=os.environ["GEMINI_API"])
    model = genai.GenerativeModel(
        model_name=_MODEL_NAME,
        system_instruction=_SYSTEM_PROMPT
    )

    user_content = json.dumps(state["structured"], ensure_ascii=False)
    start = time.perf_counter()

    try:
        response = await model.generate_content_async(
            user_content,
            generation_config=genai.GenerationConfig(
                max_output_tokens=_MAX_TOKENS,
                response_mime_type="application/json",
            )
        )
    except Exception as exc:
        logger.error("[assumption_mapper] Gemini API failed: %s", exc)
        raise RuntimeError(f"[assumption_mapper] Gemini API failed: {exc}")
```

```
    elapsed_ms = int((time.perf_counter() - start) * 1000)
    raw_content = response.text

    try:
        assumptions = json.loads(raw_content)
        if not isinstance(assumptions, list):
            raise ValueError("Expected a JSON array")
    except (json.JSONDecodeError, ValueError) as exc:
        raise ValueError(
            f"[assumption_mapper] JSON parse failed: {exc}\nRaw:
{raw_content[:300]}"
        ) from exc

    token_counts = dict(state.get("token_counts") or {})
    latencies_ms = dict(state.get("latencies_ms") or {})

    # Gemini usage metadata
    usage = response.usage_metadata
    token_counts["agent_2"] = usage.total_token_count
    latencies_ms["agent_2"] = elapsed_ms

    return {
        **state,
        "assumptions": assumptions,
        "token_counts": token_counts,
        "latencies_ms": latencies_ms,
    }
```

worker/pipeline/agents/domain_interrogator.py

```
"""
Agent 3 — Domain Interrogator.

Acts as a senior domain-expert investor for the given vertical.
Loads a vertical-specific knowledge base from disk (falls back to a
generic stub when the vertical file is not found) and generates
exactly 8 hard questions the founder has not adequately addressed.
"""

from __future__ import annotations

import asyncio
import json
import logging
import os
import time

import google.generativeai as genai
import os
```

```python
from pipeline.state import PipelineState

logger = logging.getLogger(__name__)

_MODEL_NAME = "gemini-flash-latest"
_MAX_TOKENS = 2048

# Where vertical KB markdown files live (relative to this repo's worker dir)
_VERTICALS_DIR = os.path.join(os.path.dirname(__file__), "..", "..", "verticals")

_GENERIC_KB = """\
This is a generic startup domain. Key considerations for any vertical:
- Customer acquisition costs vs lifetime value
- Competitive moat and defensibility
- Go-to-market efficiency
- Unit economics at scale
- Team execution capability
- Regulatory and compliance risks
- Market timing and readiness
"""

_SYSTEM_PROMPT_TEMPLATE = """\
You are a senior investor and domain expert in {vertical}. \
You have reviewed hundreds of pitches in this space.
You have deep knowledge of the competitive landscape, buyer behavior, \
common failure modes, and what separates successful from failed companies.

DOMAIN KNOWLEDGE:
{vertical_kb_content}

Given this startup pitch and its assumption gaps, generate the 8 hardest \
questions this founder has NOT adequately addressed — questions that any \
serious domain-expert investor would immediately ask.

For each question output:
{{ "question": str, "why_it_matters": str, "what_good_looks_like": str }}
Return a JSON array of exactly 8 objects.\
"""


def _load_vertical_kb(vertical: str) -> str:
    """Load the vertical KB markdown file; return generic fallback if absent."""
    path = os.path.join(_VERTICALS_DIR, f"{vertical}.md")
    try:
        with open(path, encoding="utf-8") as fh:
            content = fh.read().strip()
        if content:
            return content
    except FileNotFoundError:
        logger.warning("Vertical KB not found for '%s', using generic fallback.",
vertical)
```

```python
    return _GENERIC_KB


async def run_domain_interrogator(state: PipelineState) -> PipelineState:
    """
    Agent 3: Generate 8 hard domain-expert investor questions using Gemini.
    """
    vertical = state["vertical"]
    vertical_kb = _load_vertical_kb(vertical)

    system_prompt = _SYSTEM_PROMPT_TEMPLATE.format(
        vertical=vertical,
        vertical_kb_content=vertical_kb,
    )

    user_content = json.dumps(
        {
            "structured_pitch": state["structured"],
            "assumption_gaps": state["assumptions"],
        },
        ensure_ascii=False,
    )

    genai.configure(api_key=os.environ["GEMINI_API"])
    model = genai.GenerativeModel(
        model_name=_MODEL_NAME,
        system_instruction=system_prompt
    )

    start = time.perf_counter()

    try:
        response = await model.generate_content_async(
            user_content,
            generation_config=genai.GenerationConfig(
                max_output_tokens=_MAX_TOKENS,
                response_mime_type="application/json",
            )
        )
    except Exception as exc:
        logger.error("[domain_interrogator] Gemini API failed: %s", exc)
        raise RuntimeError(f"[domain_interrogator] Gemini API failed: {exc}")

    elapsed_ms = int((time.perf_counter() - start) * 1000)
    raw_content = response.text

    try:
        hard_questions = json.loads(raw_content)
        if not isinstance(hard_questions, list):
            raise ValueError("Expected a JSON array")
    except (json.JSONDecodeError, ValueError) as exc:
        raise ValueError(
```

```
            f"[domain_interrogator] JSON parse failed: {exc}\nRaw:
{raw_content[:300]}"
        ) from exc

    token_counts = dict(state.get("token_counts") or {})
    latencies_ms = dict(state.get("latencies_ms") or {})

    usage = response.usage_metadata
    token_counts["agent_3"] = usage.total_token_count
    latencies_ms["agent_3"] = elapsed_ms

    return {
        **state,
        "hard_questions": hard_questions,
        "token_counts": token_counts,
        "latencies_ms": latencies_ms,
    }
```

worker/pipeline/agents/report_synthesizer.py

```
"""
Agent 4 — Report Synthesizer.

Combines the outputs of all three prior agents into a structured,
three-section pitch-readiness report in a single JSON payload.
"""

from __future__ import annotations

import asyncio
import json
import logging
import time

import anthropic

from pipeline.state import PipelineState

logger = logging.getLogger(__name__)

_MODEL = "claude-sonnet-4-5"
_MAX_TOKENS = 1500
_RETRIES = 2
_BACKOFF_S = 2

_SYSTEM_PROMPT = """\
You are a report writer synthesizing a pitch readiness assessment. \
Combine the inputs into a clean, structured report with exactly three sections.
Return only valid JSON in this exact shape:
{
```

```
  "assumption_map": [ { "assumption": str, "category": str, "risk_level": str,
"why_unproven": str } ],
  "blind_spots": [ { "question": str, "why_it_matters": str, "what_good_looks_like":
str } ],
  "sharpening_prompts": [
    { "action": str, "rationale": str, "priority": "high"|"medium"|"low" }
  ]
}
sharpening_prompts should be 5–8 specific, actionable next steps \
derived from the assumption map and blind spots.\
"""


async def run_report_synthesizer(state: PipelineState) -> PipelineState:
    """
    Agent 4: Synthesise a three-section pitch readiness report.

    Combines structured pitch, assumptions, and hard questions from
    prior agents into a final JSON report.
    Updates state['report'] with the parsed dict.
    Records token usage in state['token_counts']['agent_4'] and
    latency in state['latencies_ms']['agent_4'].
    """
    user_content = json.dumps(
        {
            "structured_pitch": state["structured"],
            "assumptions": state["assumptions"],
            "hard_questions": state["hard_questions"],
        },
        ensure_ascii=False,
    )

    client = anthropic.AsyncAnthropic()
    start = time.perf_counter()

    response = None
    last_exc: Exception | None = None

    for attempt in range(_RETRIES + 1):
        try:
            response = await client.messages.create(
                model=_MODEL,
                max_tokens=_MAX_TOKENS,
                system=_SYSTEM_PROMPT,
                messages=[{"role": "user", "content": user_content}],
            )
            break
        except anthropic.APIStatusError as exc:
            last_exc = exc
            logger.warning("report_synthesizer attempt %d failed: %s", attempt + 1,
exc)
            if attempt < _RETRIES:
```

```
                await asyncio.sleep(_BACKOFF_S)
    else:
        raise RuntimeError(
            f"[report_synthesizer] API failed after {_RETRIES + 1} attempts:
{last_exc}"
        )

    elapsed_ms = int((time.perf_counter() - start) * 1000)
    raw_content = response.content[0].text

    try:
        report = json.loads(raw_content)
        for required_key in ("assumption_map", "blind_spots", "sharpening_prompts"):
            if required_key not in report:
                raise ValueError(f"Missing required key: {required_key}")
    except (json.JSONDecodeError, ValueError) as exc:
        raise ValueError(
            f"[report_synthesizer] JSON parse failed: {exc}\nRaw:
{raw_content[:300]}"
        ) from exc

    token_counts = dict(state.get("token_counts") or {})
    latencies_ms = dict(state.get("latencies_ms") or {})
    token_counts["agent_4"] = response.usage.input_tokens +
response.usage.output_tokens
    latencies_ms["agent_4"] = elapsed_ms

    return {
        **state,
        "report": report,
        "token_counts": token_counts,
        "latencies_ms": latencies_ms,
    }
```

worker/pipeline/state.py

```
"""
PipelineState — shared TypedDict passed through every LangGraph node.

Inputs, agent outputs, and telemetry are all stored here so each
node can read prior results and append its own.
"""

from __future__ import annotations

from typing import TypedDict


class PipelineState(TypedDict):
    # ---- Inputs -------------------------------------------------------
```

```
    raw_text: str        # Extracted text from PDF / PPTX
    vertical: str        # e.g. 'logistics_saas'
    run_id: str
    user_id: str

    # ---- Agent outputs ----------------------------------------------------
    structured: dict     # Agent 1 — structured fields extracted from deck
    assumptions: list    # Agent 2 — list of assumption dicts
    hard_questions: list # Agent 3 — list of hard question dicts
    report: dict         # Agent 4 — final synthesised report dict

    # ---- Telemetry --------------------------------------------------------
    token_counts: dict   # { 'agent_1': int, 'agent_2': int, ... }
    latencies_ms: dict   # { 'agent_1': int, 'agent_2': int, ... }
```